

Matrix FS Line Pipeline — Technical Summary

1. Purpose

Builds a debit/credit financial-statement-line crosstab ("matrix_fs_line") from a flat general ledger dataset (`mayank`), for a Tableau heatmap dashboard. Adapted from Deloitte's Management Override V2 (Cortex) logic, simplified to a single flat source instead of separate GL + Chart of Accounts files.

2. Source Data & Column Mapping

Matrix field	Source column	Derivation
journal_number	Jrn_Nmbr	direct
entity_id	LEGAENTITY	direct
is_standard	MAN_SYS_NEW	direct
account_number	Acc	direct
financial_statement_line	FS Line	direct
financial_statement_category	FS Category	direct
fiscal_year_identifier	—	literal <code>'2026'</code> (no period column in source)
user	WF_APPROVER_NAME	direct
date_posted	Creation_Date	direct
LOB	LOB	coalesced to <code>'UNSPECIFIED'</code> when null
Debit_Credit_Indicator	NET_AMOUNT_AT_REPORTING_CURRENCY	<code>D</code> if amount ≥ 0 , else <code>C</code>
debit_amount	NET_AMOUNT_AT_REPORTING_CURRENCY	amount if D, else 0

Matrix field	Source column	Derivation
credit_amount	NET_AMOUNT_AT_REPORTING_CURRENCY	abs(amount) if C, else 0
exclusion_entry	—	none — no exclusion flags exist in this source, so nothing is filtered out

3. Pipeline Stages

Stage 1 — `build_source_data(mayank)`

Row-level transform. Renames/derives every field above directly on the raw dataset, **before any aggregation**. This is deliberate: LOB, currency indicators, and the debit/credit split must exist at the finest grain so every downstream `groupBy` and `join` can key on them correctly. Doing this mapping after aggregation is what caused earlier defects (see Section 5).

Stage 2 — `build_compound_summary(source_data)`

A single `groupBy` on `((journal_number, entity_id, is_standard, fiscal_year_identifier, LOB, financial_statement_line))`, summing debit/credit amounts. **No joins, no cross-products**. This is the reconciliation anchor — its totals are guaranteed to equal the raw source totals for any given FS line, because it's a plain sum, not a join.

Stage 3 — `build_matrix_table(source_data)`

Builds the `debit_fs_line × credit_fs_line` crosstab used for the Tableau heatmap. Internally:

1. Aggregates debit rows and credit rows separately per journal.
2. Outer-joins debit aggregate to a per-journal summary key.
3. Outer-joins credit aggregate onto that result.

This produces one row per `(debit_fs_line, credit_fs_line)` combination that occurred within a journal — which is what makes the heatmap possible, but it also means **a journal with 2 debit lines and 3 credit lines produces 6 rows**, each repeating the same `debit_amount` and `credit_amount`. Summing this table directly overstates totals. This is expected and documented, not a bug — see Section 6.

Stage 4 — `reconciliation_check(...)`

Prints raw vs. compound_summary vs. matrix totals side by side, plus a per-LOB breakdown, so any variance is visible before the data reaches Tableau. Should be run every time source data changes.

4. Key Design Decisions & Why

Decision	Reason
LOB tagged at row grain in Stage 1, not joined in later	A late join risks grain mismatches and duplicate fan-out; tagging early means every downstream groupBy/join already has it.
LOB included in every join condition in <code>build_matrix_table</code> , not just groupBy	Without this, a journal spanning multiple LOBs could have a debit row from LOB-A incorrectly paired with a credit row from LOB-B — wrong pairing, not just wrong count.
<code>LOB</code> coalesced to <code>'UNSPECIFIED'</code> instead of left as null	Spark's equality join treats <code>null == null</code> as unknown, not true — so a journal with a null LOB fails to rejoin to itself, producing rows where both debit_fs_line and credit_fs_line are null despite having real line counts. This was found empirically (see Section 7) and fixed at the source.
<code>compound_summary</code> kept as a separate, non-joined table	Provides a ground-truth total unaffected by the matrix's fan-out, used both for the Python-side reconciliation check and as the Tableau denominator source for percentage calculations.
No exclusion filtering applied	This source dataset carries no exclusion flags (unlike the original Cortex script, which had zero-amount, zero-FS-impact, and account-exclusion flags) — confirmed with the business owner, documented here so it's not silently assumed later.

5. Known Structural Effect — Matrix Fan-Out

Symptom: Summing `debit_amount` or `credit_amount` directly off `matrix_table_final` returns a number larger than the true total for that FS line.

Cause: The outer join between debit rows and credit rows happens at journal grain, not at (debit_fs_line, credit_fs_line) grain. A journal with N debit lines and M credit lines produces N×M rows in the matrix. Each of those rows repeats the same debit_amount for every credit_fs_line it's paired with, and vice versa.

Resolution: Never total off the matrix table directly. Two options:

- Use `compound_summary` for any total/percentage calculation.
- In Tableau, use a `FIXED` LOD keyed to journal + fs_line to collapse duplicates before summing (see Section 8).

This is architecturally unavoidable if the matrix needs to show a debit × credit crosstab at all — it's the same tradeoff the original Deloitte Cortex script makes (their `matrix_fs_line` output has the identical property; their own `compound_summary` output exists for exactly this reason).

6. LOB Filter — Compositional Integrity

Symptom before fix: Adding LOB as a Tableau filter distorted row-count distribution (e.g. LOB A/B/C at 100/200/300 rows collapsed to 500/50/50 when filtered).

Cause: Two compounding issues:

1. LOB wasn't present at row grain when the matrix was built, so any filter/join against it operated on the wrong basis.
2. Null LOB values broke equi-joins (see Section 4), producing orphaned rows that inflated or misattributed counts.

Fix applied: LOB tagged at row grain (Stage 1) + included in every matrix join key (Stage 3) + null-coalesced to `'UNSPECIFIED'` (Section 4). With this, a plain Tableau filter on LOB only hides rows from view — it does not alter any `FIXED` LOD denominator, and it does not multiply or drop rows, because the underlying data was correct before Tableau ever touches it.

7. Defects Found During Build (chronological)

1. **Fan-out overstatement** — matrix totals ≠ `compound_summary` totals. *Root cause:* debit × credit outer join at journal grain (architectural, documented not "fixed").
2. **LOB filter distorting composition** — row counts changed shape under filter. *Root cause:* LOB added post-hoc instead of at row grain. *Fix:* moved LOB tagging into `build_source_data`.
3. **Cross-LOB pairing risk** — if a journal spans multiple LOBs, debit/credit rows from different LOBs could pair incorrectly. *Fix:* added LOB to every join condition in `build_matrix_table`.
4. **Both-null debit_fs_line / credit_fs_line rows** — found in QA screenshot, rows with real `total_lines_per_entry` but null on both sides. *Root cause:* null LOB values failing `null == null` equi-joins. *Fix:* `coalesce(LOB, 'UNSPECIFIED')` in Stage 1.

8. How to Validate the Tableau Dashboard

A. Confirm the denominator is anchored (percentage integrity)

- Create `Denominator_Check = { FIXED : SUM([Deduped Debit Amount]) }`
- Toggle any LOB filter on/off. This number must not move. If it moves, a Context/Data Source/Extract filter is touching LOB upstream of the LOD evaluation — check filter shelf placement.

B. Confirm counts/amounts aren't fan-out inflated

- Never trust raw `SUM(debit_amount) / SUM(credit_amount)` on the matrix worksheet.
- Use a Fixed LOD to dedupe before summing:

```
Deduped Debit Amount =  
{ FIXED [journal_number],[entity_id],[is_standard],  
[fiscal_year_identifier],  
[debit_fs_line] : MIN([debit_amount]) }
```

Same pattern for credit, and for `number_of_lines_debit / number_of_lines_credit` if validating counts.

C. Reconciliation worksheet (separate from the heatmap)

- Build a plain table: FS Line | SUM(Deduped Debit Amount) from matrix data | Debit amount from `compound_summary` | Variance.
- Variance should be 0 for every row. Non-zero variance = investigate join logic or a filter silently dropping records.

D. Spot-check one heatmap cell

- Right-click a cell → View Data. Note the journal_numbers feeding it.
- Pull those journal_numbers from the raw source, sum debit_amount where `Debit_Credit_Indicator = 'D'` and financial_statement_line matches. Should equal the cell.

E. Per-LOB composition check

- Compare row counts (or amounts) per LOB before and after applying the LOB filter to any *other* dimension. Composition ratios should be stable — this is the check that would have caught the original 500/50/50 collapse.

F. Filter shelf audit

- LOB filter should be a plain Filter pill.
- Confirm it is **not** in Context Filters, Data Source Filters, or Extract Filters — all three run before FIXED LOD evaluation and will re-introduce the denominator distortion this pipeline was built to prevent.

9. Open Items / Things to Watch

- If a new exclusion flag is ever added to the source data, it must be wired into `source_data` and excluded consistently in both `compound_summary` and `matrix_table`, or the two will silently diverge again.
- If `fiscal_year_identifier` moves from a hardcoded `'2026'` to a real period column, re-verify every groupBy/join still keys correctly — a wrong period grain would reintroduce the same class of fan-out/null-join issues seen with LOB.
- Recommend running `reconciliation_check()` as a standing QA step any time the source refresh changes row counts materially, not just at initial build.